Tutorial 7: PL / SQL – Guide

Section A

In this section you will write PL/SQL code without using any tables. Refer to the chapter 8 (8-7 section) and review your corresponding week 7 lecture before you start writing. Some of the code used can easily be adapted here.

Make sure to issue the following command otherwise you will not see the PL/SQL output results.

set serveroutput on; --needs to be written only once at the start of a new user session.

/* Causes Oracle client to check for and display output generated by the DBMS_OUTPUT package after each PL/SQL block, procedure, or function you execute. By default, this is off ☹ */

1a. Using the today's date, write a PL/SQL program to check whether the day is a Saturday or Sunday and accordingly display the day message.

If the day is either Saturday or Sunday then, display a message saying:

'Yay! It is weekend, being Saturday today' -- (e.g., if running your code on a Saturday)

Otherwise, display the following message:

'Oh No! It is not weekend yet, being Friday today' -- (if running your code on a Friday)

You may write an anonymous PL/SQL block here.
Make sure you declare all the variables under Declare section before you refer these in Begin...End section (e.g., slide no 9, which also uses functions sysdate, DBMS_OUTPUT.PUT_LINE and these will be useful for this task).

Need to get the value of current ***day*** from the date (e.g., Sunday – if the date is 17th Sep 2023, Tuesday – if the date is 12th or 19th Sep, etc.).

Make sure your comparisons are valid when using strings (e.g., Monday is not same as MONDAY – being case-sensitive).

Additionally, the length of the variable declared also matters (e.g., variable to store weekdays - Monday requires 6 characters; Wednesday requires 9 characters. The variable length used may be 9 and if Monday is stored in that variable, it will have additional 3 spaces). Need to keep this in mind while comparing. You can use some function to chop (trim) the additional spaces where the length is smaller than 9 for comparison to hold valid.

1b. Write a function to achieve the same result (as in 1a above). Call this function in a SQL statement or a PL/SQL block.

Should be easy to adapt to a function from 1a.'s code. Slide 28 has the syntax. Following slides have the examples.
Function must return something (i.e., have a RETURN statement to return value or expression).

2a. Write a procedure called **prc_num_addition** that requires two numbers as input and then displays their sum as output. You will need to supply two numbers when you execute the procedure.

Test whether the procedure works by issuing the following command:
exec prc_num_addition (10,20);

Named procedures are PL/SQL blocks of the code. Refer to the lecture slide 13 for the procedure syntax. Do look at the examples as well. Section 8-7b in the textbook provides many examples about stored procedures along with the screen shots.

For debugging, use the following command to see the PL/SQL errors.

show errors;

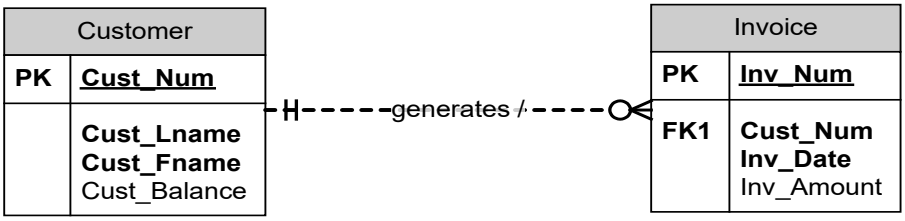
2b. Write a function to do the same (as in 2a above). Call this function in a SQL statement or a PL/SQL block.

Should be easy to adapt your 2a code to a function.

Section B

Following questions are based on the two given tables. Run the script file createpopulatepl_sqltables.sql to create and populate these tables.

Note: If you already have Customer and Invoice tables in your account, then consider renaming the pre-existing tables or alter the provided script to change the table names.



3. Write a procedure to add a new invoice record to the INVOICE table. Use the following values in the new record:

8088, 1000, 02-Apr-2022, 301.00

Name the procedure **PRC_INVOICE_ADD**.

Run a query to confirm that the record has been added.

Your named procedure will take 4 input parameters; make sure to use appropriate data types (sizes not needed for parameters) for these. Alternatively, you can use %Type data type which is based on the table structure and is preferred.

set serveroutput on size 4000;//increases the buffer size from the *default* value 2000.

4. Write a procedure to delete an invoice given the invoice number as an input parameter. Name the procedure **PRC_INV_DELETE.**

   Test the procedure by deleting invoice 8088.

   By now, you should have the skills to finish the rest (both 4 and 5)

5. Write a procedure named **PRC_CHECK_CUST_BAL** to check whether the given customer's balance is over 500.00 or not (use customer number as an input parameter in your procedure).

   If the balance is over 500.00, display a message informing the customer to urgently pay the difference between the due balance and 500 (i.e., balance minus 500 needs to be paid immediately).

   Test your procedure by issuing the following command:
   exec PRC_CHECK_CUST_BAL(1001);

   Following message will be shown (assuming 840.92 balance for customer no 1001):

   As your balance is over 500, please pay $340.92 ASAP